6CS012 - Artificial Intelligence and Machine Learning.

Assessment 2 Report

Sarcasm Detection using RNN, LSTM and LSTM+Word2Vec

Module Name	: Artificial Intelligence and Machine Learning
Module Leader	: Mr. Siman Giri
Tutor	: Mr. Rahul Parajuli
Submitted on	: 20th May, 2025
Submitted by	: Aryan Dangol
Uni ID	: 2329816
Group Name	: Byte Me AI
Team	: Srijan Limbu, Janavi Rajbhandari

Abstract

This project was executed with a goal to detect sarcasm in news headlines using deep learning modules like simple RNN, LSTM, and LSTM with pretrained Word2Vec embeddings. The dataset that was used in this project was publicly available and contained labeled sarcastic headlines, for training three separate models. The implementation comprised standard NLP preprocessing, tokenization, sequence padding, and model training with early stopping. All model evaluations were performed using accuracy, confusion matrix, and classification reports. From our evaluations, LSTM + Word2Vec model performed the best with most accuracy for predicting sarcasm in news headlines. The other two models had their own reasons as to why they underperformed.

There were challenges and difficulties like overfitting, class imbalance and so on while training the data and those challenges were solved by different techniques like dropout, early stopping, regularizer etc. After applying these techniques, we have compared those three models in terms of performance, recall, precision and accuracy. In the real world, this project can be used for multiple purposes. Since it is a part of sentiment analysis project, it can be used for fake news detection, making personalized recommendation systems, content moderation and so on.

Contents

Table of figures

1. Introduction

Sarcasm detection is an extremely difficult text classification task. Even us as humans, we sometimes have hard time understanding sarcastic humor or sarcasm. Similarly, to know if the sentence is really a sarcastic one, it really depends a lot on the tone, irony, and contextual cues. That's why most of the time it is incorrectly classified by conventional models. Sarcasm and irony are best understood within such linguistic aspects. Therefore, using deep learning methods like RNNs and LSTMs and pretrained Word2Vec embeddings, this project helps to perform sarcasm detection in news headlines.

The main goal of this project is to identify which model performs best for predicting sarcasm in news headlines. Besides that, expansion on this project can be really helpful in terms of detecting misinformation, reducing misunderstanding between AI systems and humans and many more. Some people might argue about the usage of deep models over ml models for tasks like this, not knowing that ml models does not capture semantic meanings and find them complex. Whereas, we will be using deep models like RNN, LSTM and Word2Vec and identifying which model performs the best in terms of sarcasm prediction.

2. Dataset

We used the dataset of news headlines, "Sarcastic or Not." There were 28,619 headlines in the dataset, binary labeled as 1 for sarcastic and 0 for not. We preprocessed each headline by changing it to lowercase, removed URLs, mentions, numbers, punctuation, stopwords, and lemmatized it. It was then tokenized, after which fixed padding based on the 95th percentile of sequence lengths was performed.

Source: Sarcastic Headlines Dataset (Provided by Tutor)

Description: Two columns: headline and is_sarcastic (1: sarcastic, 0: not sarcastic)

Total Records: 28,619

Preprocessing:

- Lowercasing
- Removed URLs, Mentions, Hashtags, Punctuation, Numbers
- Expanded Contractions
- Stopwords removed, Lemmatization using NLTK
- Tokenization with Keras Tokenizer
- Padding using the 95th percentile of sequence lengths

3. Methodology

3.1 Text Preprocessing:

Text preprocessing contains certain steps that are done to ensure that the data is cleaned and ready to be fed into the model. Text preprocessing vastly decreases the noise in the dataset and prevents unnecessary waste of resources. In our project we have these steps to preprocess the text.

a. Uppercase to lowercase conversion
b. Removal of words that do not have meanings such as URLs, @, #, numbers etc.
c. Stop words removal
d. Tokenization and lemmatization
e. Labels converting
f. Applying padding using 95^{th} percentile length

3.2. Model Architecture

We have 3 model architectures which are described below with their embedding layer and activation function used.

- **Model 1: Simple RNN**

In Simple RNN(Recurrent Neural Network), the model keeps the memory of the words in a compressed format and is updated per time step. This model has a short-term memory and its memory is limited to a standard 20 words. That means, in most of the case, RNN starts to give inaccurate predictions if the sentence is longer than 20 words. During the training of our model as well, RNN started performing badly as soon as it starts to analyze longer sentences. Therefore, it is where the vanishing gradient problem of RNN comes from. Vanishing gradient means when the gradient is too small and the model stops learning or becomes very slower. It occurs during back propagation if the gradient of the loss is less and becomes lesser with each propagation.

- **Model 2: LSTM**

LSTM(Long Short Term Memory) is a deep learning model that solves the vanishing gradient problem of RNN. In this model, there are three gates: input, output and forget gate with a memory state and a cell state. While processing a sentence, the gates process input parallelly. The forget gate decides which words to keeps and which ones to discard. The input gate decides which words to be added to the old state. The output gate then decides the part of the cell state to output. Therefore, this model solves the vanishing gradient problem by keeping the cell state as it is until the new state is deemed as more important as the old state. And this helps by preserving gradients and long-term dependencies of words in a sentence.

- **Model 3: LSTM + Word2Vec**

Simply, pretrained Word2Vec in the deep learning model is like a dictionary that has all the possible meanings of a word. So why do we need Word2Vec embedding in the LSTM? Although LSTM performs better than RNN and actually keeps the long-term dependencies in a sentence, LSTM as an individual model cannot understand the sematic relationship and the context of a word in a sentence. This causes the model to falsely predict whether the sentence is sarcastic or not. Therefore, with the help of Word2Vec, each word now gets vectorized into 300-dimension vector and while prediction, the model looks up the vectors to see which feature is closer to the word and predicts accordingly.

3.3. Training Setup

For training the models, the following were used.

- Loss Function: Binary Cross entropy because it is a multi-class classification problem
- Optimizer: Adam because it uses AdaGrad and RMSProp for better optimization.
- Early Stopping (patience=2): This helps in keeping the model from overfitting and patience 2 means, if there is no improvement within the two epochs in terms of validation data the model stops to prevent overfitting.
- Epochs: 10. This means the model is trained for over 10 times.
- Batch Size: 64. Keeping the batch size 64 is a standard. This means there are 64 training samples used to update the weights. If it is more than 64, the training is

faster but the model might overfit and if it is smaller, the training is slower but there are less data to learn from so it might not understand complex relationships.

- Validation Split: 0.2. This means the data is spit by 80/20 where 80% is used for training and 20% for validation.

4. Experiments and Results

4.1 RNN vs LSTM Performance

Metric	Simple RNN	LSTM
Accuracy	80.35%	80.01%
Macro F1	0.81	0.82
Precision	0.82	0.84
Recall	0.81	0.87

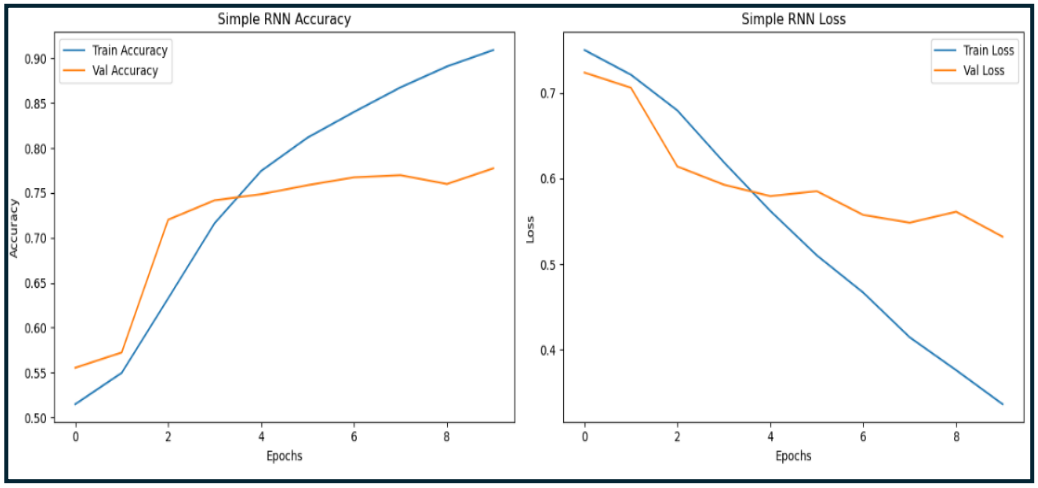

Figure 1: Loss and Accuracy of Train and Validation of RNN model

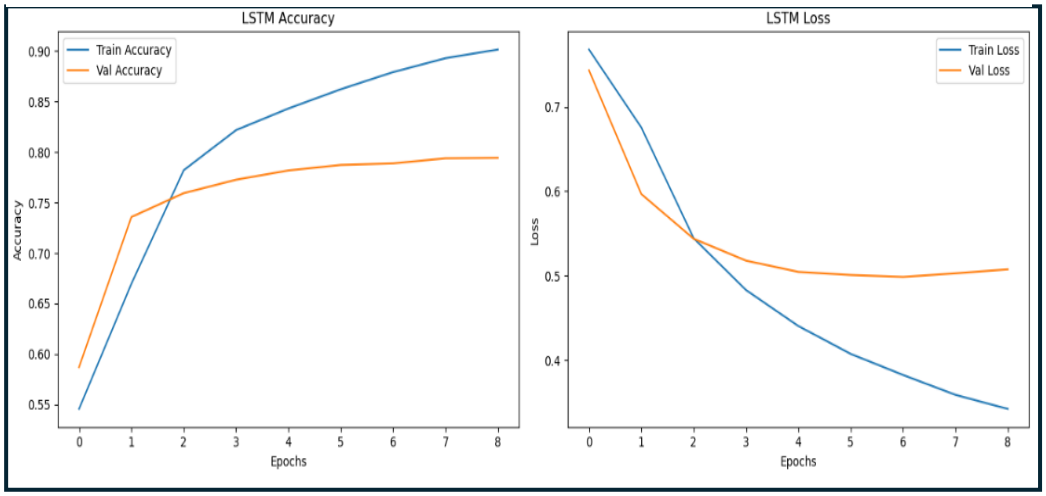

Figure 2: Loss and Accuracy of Train and Validation of LSTM model

Observation: Simple RNN unexpectedly performed slightly better than LSTM on this dataset. While LSTM typically handles longer dependencies better, the short length of headlines may have favored RNN's faster architecture. On the flip side, precision and recall value of RNN is not so good as LSTM. The precision shows out of how many positive predictions were actually correct and recall shows out of the actual positive values how many the model could predict as positive. So, having higher precision and recall in LSTM over RNN means the LSTM is performing better in terms of predicting the actual positive and catches more true predictions than RNN.

4.2 Computational Efficiency

Model	Training Time	Parameters (approx.)	Notes
Simple RNN	Fastest	Fewest	Low memory usage
LSTM	Medium	Moderate	More stable learning
LSTM + Word2Vec	Slowest	Highest	Longer training time

Observation:

Simple RNN was the fastest to train and least memory-intensive. LSTM required more time but offered better stability. LSTM + Word2Vec took the longest due to its larger embedding size and frozen weights.

4.3 Training with Different Embeddings

We evaluated the impact of using:

- Random Trainable Embeddings (RNN & LSTM)
- Pretrained Word2Vec Embeddings (LSTM + Word2Vec)

Model	Embedding Type	Accuracy	F1 Score
LSTM	Trainable (Random)	80.01%	0.77
LSTM + Word2Vec	Pretrained (frozen)	81.85%	0.82

Observation:

Word2Vec embeddings slightly improved performance. This is because while using random embeddings, the model does not know the meaning of the word and their context until its trained multiple times. And even then, the model might memorize the meaning and not actually learn. The semantic relationship is not properly known while using random embeddings. Whereas, a pretrained model already has 300-dimension vector of each word which we can say as 300 interpretations of a single word depending on the context. Therefore, a model using pre trained Word2Vec already knows which word holds the most importance in the context of the sentence. Therefore, catching the semantic relationship between the words, making a correct prediction. The model using the Word2Vec embeddings achieved the highest accuracy and F1, confirming that pretrained embeddings help even without fine-tuning.

4.4 Model Evaluation

We used standard evaluation metrics for binary classification. And the results are as follows:

Model	Accuracy	Precision	Recall	F1 Score
Simple RNN	80.35%	0.82	0.80	0.81
LSTM	80.01%	0.84	0.72	0.77
LSTM + Word2Vec	81.85%	0.85	0.80	0.82

The model evaluation shows that the best performing model is actually LSTM+Word2Vec with 81.85% accuracy. Even on other metrics like precision, recall and f1 score. We can say that it was expected because RNN has a vanishing gradient problem and does not have a good memory as it keeps forgetting the earlier dependencies once the input gets longer. LSTM does not catch meaningful relationships between the words but it has long term memories unlike RNN. And LSTM + Word2Vec has the combination of longer word dependency and conceptual meaning of words in a sentence. Therefore, between these three models, LSTM with Word2Vec embeddings has the precision and recall showing that the model correctly predicts and is best for catching actual correct predictions. In comparision, LSTM missed more of the correct predictions due to low recall but there were less misclassifications than RNN.

4.5 Evaluation Metrics

We used the three evaluation metrics, for comparing the performance of the models.

- Accuracy:
  Calculated how accurate the predictions were across three models. And according to the table LSTM + Word2Vec was the most accurate. Followed by simple RNN and LSTM. Since LSTM works best with longer sentences, we can say that there were more shorter sentences than longer ones according to the accuracy which is more in RNN than LSTM.

- Precision, recall, F1-Score
  These three calculated the accuracy of the predictions(precision), number of positive predictions out of all the positives(recall) and performance score(F1 score). The higher the precision, the better will be the accuracy of prediction. And the higher the recall the better it is at catching real positives. Therefore, we can tell by the diagram that LSTM+Word2Vec is the best model for prediction.

Simple RNN:

	precision	recall	f1-score	support
0	0.82	0.80	0.81	2995
1	0.79	0.81	0.80	2729
accuracy			0.80	5724
macro avg	0.80	0.80	0.80	5724
weighted avg	0.80	0.80	0.80	5724

Figure 3: Evaluation metrics of RNN

LSTM:

	precision	recall	f1-score	support
0	0.77	0.87	0.82	2995
1	0.84	0.72	0.77	2729
accuracy			0.80	5724
macro avg	0.81	0.80	0.80	5724
weighted avg	0.80	0.80	0.80	5724

Figure 4: Evaluation metrics of LSTM

LSTM + Word2Vec:

	precision	recall	f1-score	support
0	0.85	0.80	0.82	2995
1	0.79	0.84	0.82	2729
accuracy			0.82	5724
macro avg	0.82	0.82	0.82	5724
weighted avg	0.82	0.82	0.82	5724

Figure 5: Evaluation metrics of LSTM + Word2Vec

- Confusion Matrix:

  According to the confusion matrix:

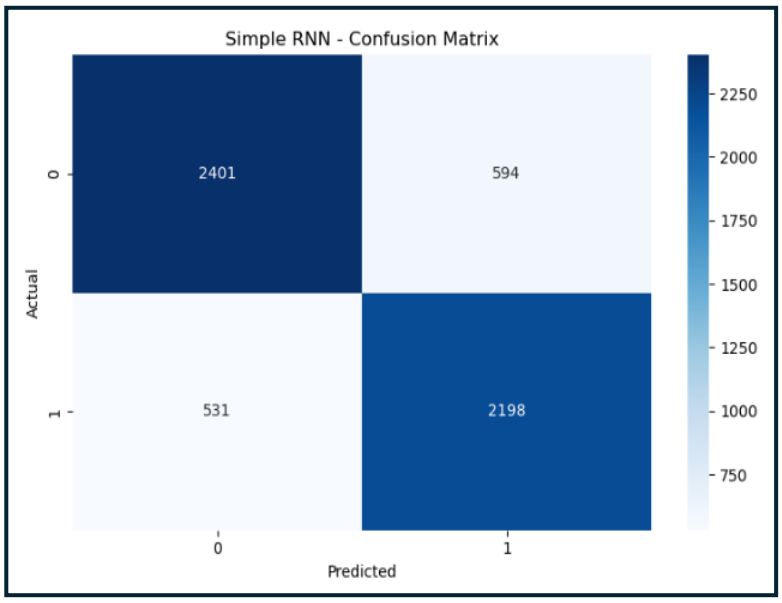

Figure 6: Confusion matrix of RNN

In case of RNN, it correctly predicts 2401 cases of non-sarcastic news headlines(class 0) and 2198 cases of sarcastic news headlines(class 1). Also, the model incorrectly predicts 594 cases of class 0 as class 1. Therefore, we can see that this model has recall a little higher than precision.

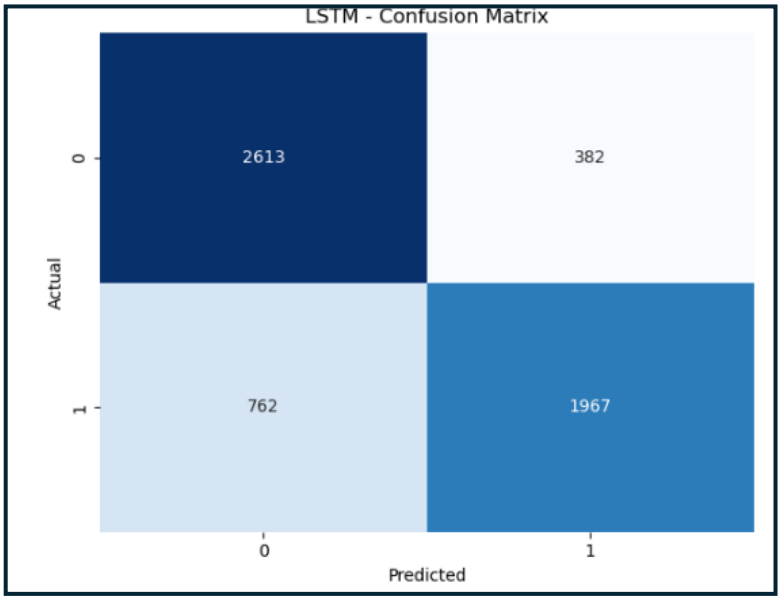

Figure 7: : Confusion matrix of LSTM

In the case of LSTM, it correctly predicts 2613 cases of non-sarcastic news headlines(class 0) and 1967 cases of sarcastic news headlines(class 1). Also, the model incorrectly predicts 382 cases of class 0 as class 1. Therefore, we can see that this model has precision higher than recall. So, if this model predicts a sentence as sarcastic, it is mostly sarcastic.

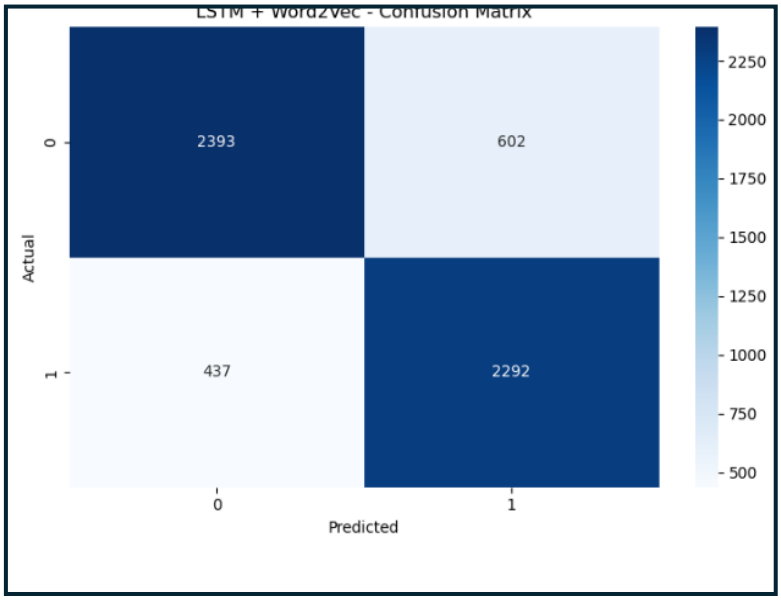

Figure 8: : Confusion matrix of LSTM+Word2Vec

In the case of LSTM+ Word2Vec, it correctly predicts 2393 cases of non-sarcastic news headlines(class 0) and 2292 cases of sarcastic news headlines(class 1). Also, the model incorrectly predicts 602 cases of class 0 as class 1. Therefore, we can see that this model has recall higher than precision. That means the model does better at not missing sarcasm than accurately predicting actual sarcasm.

4.6 Observations

- The simple RNN exceeded our wildest expectations with an average accuracy of 80.35% and a macro F1-score of 0.81. Despite its simpler architecture, this model effectively captured short-term dependencies in headline text.
- Making a mild comparison, LSTM clocked slightly below the performance of simple RNN, giving an accuracy of 80.01% and a macro F1 score of 0.77.
  This might be because of overfitting. Overfitting might be due to noise in the data or because of variability introduced during training. However, the behavior is consistent and shows strength in sequential text classification.
- This version had the most accurate Word2Vec in LSTM, which was 81.85%, and with the highest macro F1, which was 0.82. The generalization capability in improving precision regarding class detection for sarcastic comments was very strong. This shows that pretrained embeddings would give semantically rich word representations to enhance performance.
- The models showed really good performances, yet there was a slight edge to Word2Vec. Although somewhat negligible, differences in performance are arguably remarkable, especially with respect to precision and recall trade-offs across classes.

5. GUI for real time predictions

An easy-to-use interface was designed using Gradio that can predict real-time sarcasm in any given headline. LSTM + Word2Vec was deployed as the best-fitting model for adhering to the requirements of the assignment. The GUI displays the confidence score.

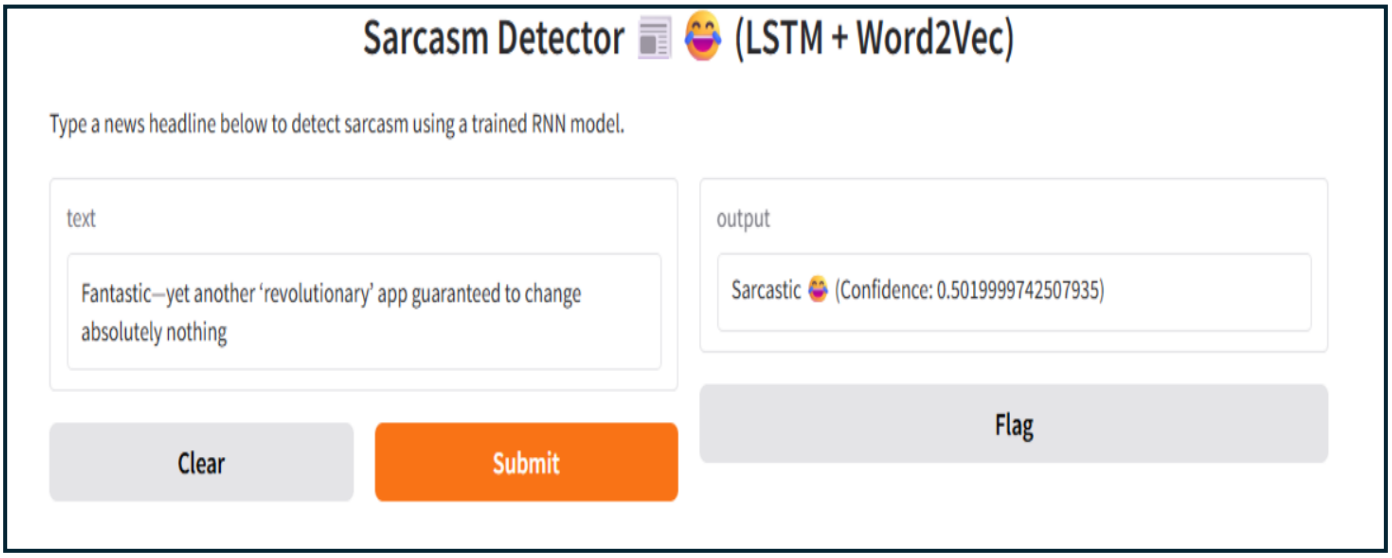

Figure 9: GUI of sarcasm detector detecting sarcastic sentence

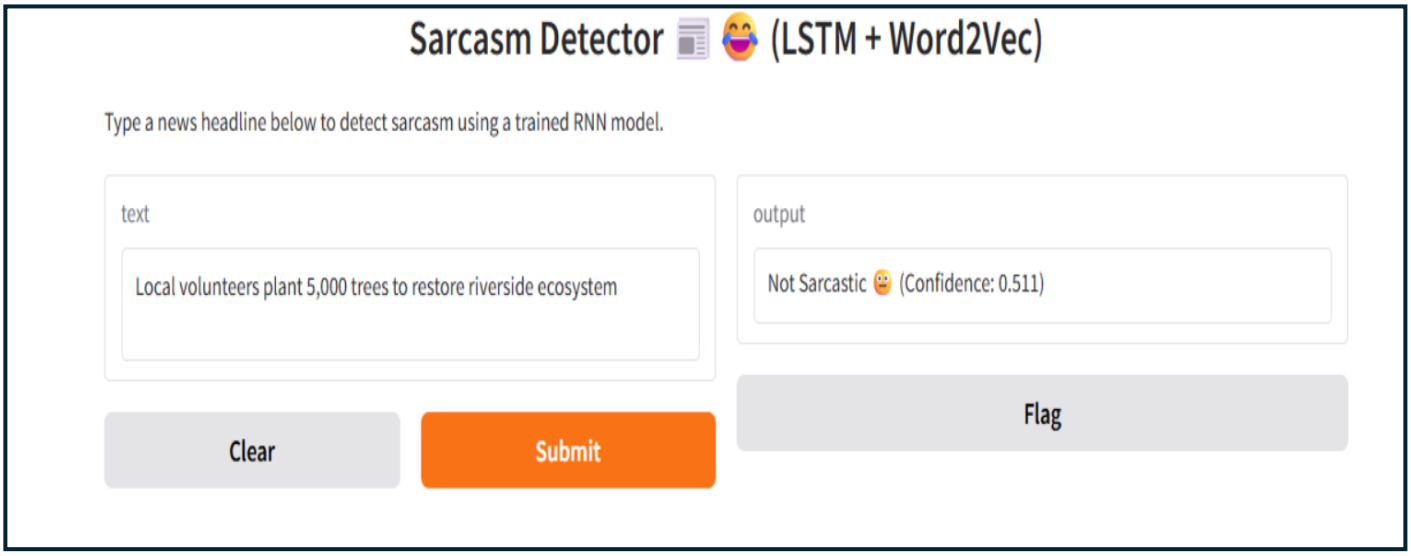

Figure 10: GUI of sarcasm detector detecting non sarcastic sentence

6. Conclusion and Future Work

Hence, in the short headlines, any of the provided models such as Simple RNN or LSTM or LSTM + Word2Vec may very well detect sarcasm. Surprisingly, the performance of the Simple RNN was nearly the same as the LSTM model, while the LSTM + Word2Vec model reached the highest accuracy and macro F1-score, thereby justifying the use of the pre-trained embeddings.

For future work:

- Train embeddings on more specialized fields to have a better performing model.
- Trying out transformer based architectures like BERT
- Much more extensive and diverse datasets to generalize better
- Fine-tuning Word2Vec instead of freezing it.